

Student

I keep hearing about stored procedures in data engineering pipelines. What exactly are they and how are they different from just writing SQL queries?

ChatGPT • AI Assistant

CONCEPT CARD

A **stored procedure** and a **user-defined function (UDF)** are both named, reusable blocks of SQL logic stored inside the database itself — not in your application code. But they serve very different purposes.

The core distinction:

- A **Function** returns a value. You can call it inside a SELECT, WHERE, or expression — it behaves like a built-in SQL function.
- A **Stored Procedure** performs an action. You call it with CALL or EXEC, it can manage transactions (BEGIN/COMMIT/ROLLBACK), and it can contain complex control flow.

🗨️ ANALOGY

A function is like a calculator — you put a number in, get a computed result back. You can embed a calculator in the middle of any arithmetic expression. A stored procedure is like a workflow script — you trigger it, it runs a sequence of operations, modifies data, logs results, handles errors, and finishes. You would not embed a workflow script inside a SELECT clause.

🌐 REAL WORLD APPLICATION

In data engineering, stored procedures are used for: SCD Type 2 merge operations, nightly batch status updates, audit logging, bulk deactivation workflows, and any operation that must execute as a single atomic transaction. Functions are used for: classification logic (customer tier, discount percentage), derived column calculations, and reusable filtering conditions.

Student

What schema are we using?

CONCEPT CARD

Same e-commerce schema, extended with a few new tables for this section:

- `customers(customer_id, name, email, city, signup_date, tier, is_active)`
- `orders(order_id, customer_id, order_date, total_amount, status)`
- `products(product_id, name, category_id, price, stock_qty, stock_status)`
- `categories(category_id, name)`
- `order_summary_daily(summary_date, total_orders, total_revenue, created_at)`
- `customer_deactivation_log(log_id, customer_id, deactivated_at, reason)`

USER-DEFINED FUNCTION

1

Returns: a single value or table

→f

Called from: SELECT clause, WHERE clause, expressions

✗

Transaction control: No — cannot COMMIT or ROLLBACK inside a function

🔗

Use cases: scalar computation (tier classification, discount %), table-valued results, reusable expressions

SELECT name, get_customer_tier(total_spend) AS tier FROM customers;

STORED PROCEDURE

→📄

Returns: nothing, or result sets via OUT parameters

CALL

Called from: CALL procedure_name() or EXEC procedure_name

↺↻

Transaction control: Yes — can BEGIN, COMMIT, ROLLBACK, handle exceptions

💰🔗

Use cases: batch workflows, SCD merges, audit logging, status updates, multi-step transactions

CALL process_daily_orders();

Key rule: if you need it in a SELECT expression → function. If it modifies data or manages transactions → stored procedure.

Function vs Stored Procedure — Core Distinction

Function vs Stored Procedure — Quick Reference		
Property	User-Defined Function	Stored Procedure
Returns	A value (scalar) or a result set (table-valued)	Nothing, or result sets via OUT parameters
Called from	SELECT, WHERE, expression context	CALL proc_name() or EXEC proc_name

file:///C:/Users/Abhiram Chittampally/Documents/ACC/Data-Model-Engineering-Foundation-SQL(Structured-Query-Language).html

2/48

Property	User-Defined Function	Stored Procedure
Transaction control	No — cannot COMMIT or ROLLBACK	Yes — BEGIN, COMMIT, ROLLBACK, SAVEPOINT
Side effects (DML)	No — functions are generally read-only	Yes — INSERT, UPDATE, DELETE allowed
Exception handling	Limited — raises an error	Full — TRY/CATCH or EXCEPTION blocks
Used in Airflow/scheduler	Indirectly (called in SQL task)	Directly — CALL proc_name() is the task
Version control	Stored in DB — harder to track	Stored in DB — harder to track
Modern alternative	dbt macros, Python UDFs	dbt models, Python scripts, dbt snapshots

sql – Function and Procedure Syntax Templates – PostgreSQL

```
-- SCALAR FUNCTION template
CREATE OR REPLACE FUNCTION function_name(param1 TYPE, param2 TYPE)
RETURNS return_type
LANGUAGE plpgsql
AS $$
DECLARE
    variable_name TYPE;
BEGIN
    -- logic here
    RETURN some_value;
END;
$$;

-- TABLE-VALUED FUNCTION template (RETURNS TABLE)
CREATE OR REPLACE FUNCTION function_name(param TYPE)
RETURNS TABLE(col1 TYPE, col2 TYPE)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
        SELECT col1, col2 FROM some_table WHERE condition;
END;
```

```

$$;

-- STORED PROCEDURE template
CREATE OR REPLACE PROCEDURE procedure_name(param TYPE)
LANGUAGE plpgsql
AS $$
DECLARE
    v_variable TYPE;
BEGIN
    -- actions: INSERT, UPDATE, DELETE
    COMMIT;
EXCEPTION WHEN OTHERS THEN
    ROLLBACK;
    RAISE;
END;
$$;

-- Call syntax
SELECT function_name(arg);           -- function in SELECT
CALL procedure_name(arg);            -- procedure

```

SCALAR FUNCTIONS — RETURNING A SINGLE COMPUTED VALUE

ChatGPT • AI Assistant

A left-to-right flow diagram. Left box (blue, label 'Function Input'): shows parameter box 'total_spend = 84200.00' flowing into a function block. Center box (white, label 'Function Body — get_customer_tier'): shows the conditional logic as a decision diamond tree: 'total_spend >= 50000?' YES → 'RETURN Gold' (green box). NO → next diamond 'total_spend >= 10000?' YES → 'RETURN Silver' (yellow box). NO → 'RETURN Bronze' (orange box). Right box (green, label 'Return Value'): shows the returned text value 'Gold' being inserted into a SELECT query result column labeled 'tier'. Below the flow, a usage example strip shows the function embedded in three contexts: 'SELECT get_customer_tier(spend) — in SELECT column', 'WHERE get_customer_tier(spend) = Gold — in WHERE clause', 'HAVING get_customer_tier(SUM(spend)) = Silver — in HAVING'. Clean left-to-right flow, decision tree in center, consistent color coding.

Scalar Function Flow — Input Parameter to Return Value

sql — Scalar Functions — Tier Classifier, Discount Calculator, and Usage

```

-- Scalar function: customer tier classifier
CREATE OR REPLACE FUNCTION get_customer_tier(total_spend NUMERIC)
RETURNS TEXT
LANGUAGE plpgsql
AS $$

```

```
BEGIN
    IF total_spend IS NULL OR total_spend < 0 THEN
        RETURN 'Bronze'; -- safe default for NULL / negative
    ELSIF total_spend ≥ 50000 THEN
        RETURN 'Gold';
    ELSIF total_spend ≥ 10000 THEN
        RETURN 'Silver';
    ELSE
        RETURN 'Bronze';
    END IF;
END;
$$;

-- Use in SELECT to classify all customers
SELECT c.customer_id,
       c.name,
       c.city,
       COALESCE(SUM(o.total_amount), 0) AS
lifetime_spend,
       get_customer_tier(COALESCE(SUM(o.total_amount), 0)) AS tier
FROM   customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name, c.city
ORDER BY lifetime_spend DESC;

-- Use in HAVING to filter by tier
SELECT c.name, SUM(o.total_amount) AS lifetime_spend
FROM   customers c JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name
HAVING get_customer_tier(SUM(o.total_amount)) = 'Gold';

-- Discount function: two parameters
CREATE OR REPLACE FUNCTION calculate_discount(
    order_total   NUMERIC,
    customer_tier TEXT
)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $$
DECLARE
    v_discount NUMERIC := 0;
BEGIN
    v_discount := CASE
        WHEN order_total ≥ 5000 THEN 10
        WHEN order_total ≥ 2000 THEN 5
        ELSE 0
    
```

```
END;  
v_discount := v_discount + CASE customer_tier  
    WHEN 'Gold' THEN 5  
    WHEN 'Silver' THEN 2  
    ELSE 0  
END;  
RETURN v_discount;  
END;  
$$;  
  
-- Apply in pricing query  
SELECT o.order_id, o.total_amount, c.tier,  
       calculate_discount(o.total_amount, c.tier) AS discount_pct,  
       o.total_amount * (1 - calculate_discount(o.total_amount, c.tier)  
       / 100.0)  
                                     AS discounted_total  
FROM orders o JOIN customers c ON o.customer_id = c.customer_id  
WHERE o.status = 'pending';
```

Student

Let us start with the simplest case — a scalar function. You mentioned a customer tier classifier. How do I write that?

ChatGPT • AI Assistant

CONCEPT CARD

A **scalar function** takes one or more input parameters and returns exactly one value. Once created, you call it like any built-in SQL function — inside SELECT, WHERE, CASE, or any expression context.

EXAMPLE CARD**Example 1 — Scalar function: classify customers by total spend**

sql — Code Example

```
CREATE OR REPLACE FUNCTION get_customer_tier(total_spend NUMERIC)  
RETURNS TEXT  
LANGUAGE plpgsql  
AS $$  
BEGIN
```

```
IF total_spend ≥ 50000 THEN
    RETURN 'Gold';
ELSIF total_spend ≥ 10000 THEN
    RETURN 'Silver';
ELSE
    RETURN 'Bronze';
END IF;
END;
$$;
```

Once created, use it in any SELECT:

sql – Code Example

```
-- Classify every customer by their lifetime spend
SELECT c.customer_id,
       c.name,
       c.city,
       SUM(o.total_amount) AS
lifetime_spend,
       get_customer_tier(SUM(o.total_amount)) AS tier
FROM   customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name, c.city
ORDER BY lifetime_spend DESC;
```

Output (sample):

code – Code Example

customer_id	name	city	lifetime_spend	tier
101	Amelia Watson	Mumbai	84200.00	Gold
102	Rohan Mehta	Delhi	22500.00	Silver
205	Sara Khan	Hyd	3100.00	Bronze
312	Priya Nair	Mumbai	0.00	Bronze

Example 2 — Use the function in WHERE to filter only Gold customers:

sql – Code Example

```
SELECT c.name, SUM(o.total_amount) AS lifetime_spend
FROM   customers c
JOIN   orders o ON c.customer_id = o.customer_id
```

```
GROUP BY c.customer_id, c.name
HAVING get_customer_tier(SUM(o.total_amount)) = 'Gold';
```

Example 3 — Scalar function with multiple parameters:

sql – Code Example

```
CREATE OR REPLACE FUNCTION calculate_discount(
    order_total NUMERIC,
    customer_tier TEXT
)
RETURNS NUMERIC
LANGUAGE plpgsql
AS $$
DECLARE
    v_discount NUMERIC := 0;
BEGIN
    -- Base discount by order size
    IF order_total ≥ 5000 THEN
        v_discount := 10;
    ELSIF order_total ≥ 2000 THEN
        v_discount := 5;
    ELSE
        v_discount := 0;
    END IF;

    -- Tier bonus
    IF customer_tier = 'Gold' THEN
        v_discount := v_discount + 5;
    ELSIF customer_tier = 'Silver' THEN
        v_discount := v_discount + 2;
    END IF;

    RETURN v_discount; -- returns discount percentage
END;
$$;

-- Use in a pricing query:
SELECT o.order_id,
       o.total_amount,
       c.tier,
       calculate_discount(o.total_amount, c.tier) AS discount_pct,
       o.total_amount * (1 - calculate_discount(o.total_amount,
c.tier) / 100.0)
AS
discounted_total
```

```
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status = 'pending';
```

Student

What language is the function body written in? Is it always PL/pgSQL?

ChatGPT • AI Assistant

The function body language depends on the database:

- **PostgreSQL:** PL/pgSQL (procedural SQL), PL/Python, PL/Perl, or pure SQL for simple expressions.
- **MySQL:** SQL/PSM (its own procedural dialect).
- **SQL Server:** T-SQL.
- **Snowflake:** JavaScript, Python, Java, Scala, or SQL.

For simple scalar functions with no loops or conditionals, you can use `LANGUAGE sql` instead of `plpgsql` :

sql – Code Example

```
-- Simple SQL function (no procedural logic needed)
CREATE OR REPLACE FUNCTION price_with_gst(price NUMERIC)
RETURNS NUMERIC
LANGUAGE sql
AS $$
    SELECT price * 1.18;
$$;
```

⚠ WARNING

Performance consideration: scalar functions called row-by-row can be slow on large tables — the function body executes once per row. For large analytical queries, consider rewriting the function logic as a CASE expression directly in the query:

sql – Code Example

```
-- Function call (slower on 10M rows):
SELECT name, get_customer_tier(lifetime_spend) FROM customer_summary;

-- Inline CASE equivalent (faster – evaluated in one pass):
```

```
SELECT name,  
       CASE WHEN lifetime_spend ≥ 50000 THEN 'Gold'  
            WHEN lifetime_spend ≥ 10000 THEN 'Silver'  
            ELSE 'Bronze'  
       END AS tier  
FROM customer_summary;
```

Use functions for readability and reuse. Be aware of per-row call overhead on very large datasets.

INTERVIEW CARD

Frequency: Medium-High

Question: What is a scalar UDF? What are the performance considerations when using one on large tables?

Expected Answer: A scalar user-defined function takes parameters and returns a single value. It can be used anywhere a SQL expression can appear — SELECT, WHERE, HAVING, ORDER BY. Performance consideration: the function body executes once per row in the result set, which can cause significant overhead on large tables because each call has function invocation overhead. For analytical queries on millions of rows, inlining the equivalent CASE expression is often faster. Functions are best suited for readability, reuse across queries, and logic that changes — where a single update to the function propagates everywhere it is called.

Follow-up: What is the difference between LANGUAGE sql and LANGUAGE plpgsql for a function? (Answer: LANGUAGE sql functions are pure SQL expressions — simpler, faster, and inlined by the optimizer. LANGUAGE plpgsql functions support procedural constructs: IF/ELSIF, loops, DECLARE variables, EXCEPTION handling — needed for complex conditional logic.)

PRACTICE QUIZ

1. Can a scalar function be used in a WHERE clause?
2. What is the performance risk of calling a scalar function on a 10-million-row table?
3. Write the return statement for a function that returns 'Premium' if price > 1000, else 'Standard'.
4. Predict: `SELECT get_customer_tier(NULL)` — what does this return if the function has no NULL check?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. Yes — a scalar function returns a value and can appear in any expression context including WHERE.
2. The function executes once per row — 10 million function invocations. Each has overhead. For large analytical queries, inlining as a CASE expression is faster.
3. `IF price > 1000 THEN RETURN 'Premium'; ELSE RETURN 'Standard'; END IF;`
4. It depends on the function logic — if `totalspend` is `NULL`, `'NULL' >= 50000` evaluates to `NULL` (not `TRUE`), so `ELSIF` and `ELSE` would eventually fire, returning `'Bronze'`. A robust function should add `'IF totalspend IS NULL THEN RETURN 'Bronze'; END IF;'` at the top.

❓ CHECKPOINT QUESTION

Write a scalar function `get_stock_status(stock_qty INT) RETURNS TEXT` that returns 'Out of Stock' for 0, 'Low Stock' for 1-10, 'In Stock' for 11-100, and 'Well Stocked' for above 100. Then write a `SELECT` query that uses it to show all products with their name, price, stockqty, and computed stockstatus.

TABLE-VALUED FUNCTIONS — RETURNING A RESULT SET

ChatGPT • AI Assistant

A three-column comparison diagram. Column 1 (gray background, label 'Regular View'): shows `CREATE VIEW` with fixed `WHERE category_id = 1`. Usage: `SELECT * FROM electronics_view`. Limitation badge: 'Fixed logic — one view per category. No parameters.' Column 2 (green background, label 'Table-Valued Function'): shows `CREATE FUNCTION get_products_by_category(p_category_id INT)`. Usage: `SELECT * FROM get_products_by_category(1)`, `SELECT * FROM get_products_by_category(2)`. Benefit badge: 'Parameterised — one function, any category. Reusable.' Column 3 (blue background, label 'Inline Subquery'): shows `FROM (SELECT ... WHERE category_id = 1) p`. Limitation badge: 'One-off — must rewrite in every query. Not reusable.' Below the three columns, a decision strip reads: 'Same logic used in one place? → subquery. Same result every time? → view. Same logic with different parameters? → table-valued function.' Clean three-column layout, consistent color scheme.

Table-Valued Function vs View vs Subquery

sql — Table-Valued Function — Products by Category with Stock Status

```
-- Table-valued function returning a result set
CREATE OR REPLACE FUNCTION get_products_by_category(p_category_id INT)
RETURNS TABLE(
    product_id    INT,
    product_name  TEXT,
```

```
price          NUMERIC,
stock_qty      INT,
stock_status   TEXT
)
LANGUAGE plpgsql
AS $$
BEGIN
    RETURN QUERY
        SELECT p.product_id,
               p.name::TEXT,
               p.price,
               p.stock_qty,
               CASE
                   WHEN p.stock_qty = 0    THEN 'Out of Stock'
                   WHEN p.stock_qty ≤ 10  THEN 'Low Stock'
                   WHEN p.stock_qty ≤ 100 THEN 'In Stock'
                   ELSE                    'Well Stocked'
               END::TEXT
        FROM   products p
        WHERE  p.category_id = p_category_id
        ORDER BY p.price DESC;
END;
$$;

-- Basic usage in FROM
SELECT * FROM get_products_by_category(1); -- Electronics
SELECT * FROM get_products_by_category(2); -- Fitness

-- TVF used in a JOIN
SELECT p.product_name, p.stock_qty, p.stock_status,
       COUNT(oi.order_id) AS pending_orders
FROM   get_products_by_category(1) p
JOIN   order_items oi ON p.product_id = oi.product_id
JOIN   orders o ON oi.order_id = o.order_id
WHERE  p.stock_status IN ('Low Stock', 'Out of Stock')
       AND o.status = 'pending'
GROUP BY p.product_name, p.stock_qty, p.stock_status
ORDER BY p.stock_qty ASC;

-- Materialise for expensive TVFs called multiple times
CREATE TEMP TABLE tmp_electronics AS
    SELECT * FROM get_products_by_category(1);
-- Use tmp_electronics in multiple subsequent queries
SELECT * FROM tmp_electronics WHERE stock_status = 'Low Stock';
```

```
SELECT COUNT(*) FROM tmp_electronics WHERE price > 1000;  
DROP TABLE tmp_electronics;
```

Student

Can a function return a whole table of rows instead of just one value?

ChatGPT • AI Assistant

CONCEPT CARD

Yes — a **table-valued function (TVF)** returns a result set (a table). You use it in the FROM clause exactly like a regular table or subquery. It is essentially a parameterised view — a view that accepts arguments.

sql – Code Example

```
-- Calling a table-valued function  
SELECT * FROM get_products_by_category(3);  
  
-- Join a TVF with another table  
SELECT p.*, o.order_count  
FROM get_products_by_category(3) p  
JOIN product_order_counts o ON p.product_id = o.product_id;
```

EXAMPLE CARD**Example 1 — TVF: Return all products in a category with stock status**

sql – Code Example

```
CREATE OR REPLACE FUNCTION get_products_by_category(p_category_id INT)  
RETURNS TABLE(  
    product_id INT,  
    product_name TEXT,  
    price NUMERIC,  
    stock_qty INT,  
    stock_status TEXT  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
    RETURN QUERY
```

```
SELECT p.product_id,
       p.name::TEXT,
       p.price,
       p.stock_qty,
       CASE
         WHEN p.stock_qty = 0      THEN 'Out of Stock'
         WHEN p.stock_qty ≤ 10    THEN 'Low Stock'
         WHEN p.stock_qty ≤ 100   THEN 'In Stock'
         ELSE                      'Well Stocked'
       END::TEXT AS stock_status
FROM   products p
WHERE  p.category_id = p_category_id
ORDER BY p.price DESC;

END;
$;$
```

Usage:

sql – Code Example

```
-- Get all Electronics products with stock status
SELECT * FROM get_products_by_category(1); -- category_id 1 =
Electronics

-- Output:
-- product_id | product_name      | price   | stock_qty | stock_status
-- -----
-- 12         | Laptop Pro       | 3499.00 | 45        | In Stock
-- 8          | Wireless Mouse   | 799.00  | 8         | Low Stock
-- 31         | USB Hub          | 299.00  | 0         | Out of Stock
```

Example 2 — TVF used in a JOIN:

sql – Code Example

```
-- Find low-stock Electronics products that have pending orders
SELECT p.product_name,
       p.stock_qty,
       p.stock_status,
       COUNT(oi.order_id) AS pending_orders
FROM   get_products_by_category(1) p
JOIN   order_items oi ON p.product_id = oi.product_id
JOIN   orders o ON oi.order_id = o.order_id
WHERE  p.stock_status IN ('Low Stock', 'Out of Stock')
AND    o.status = 'pending'
```

```
GROUP BY p.product_name, p.stock_qty, p.stock_status  
ORDER BY p.stock_qty ASC;
```

Example 3 — TVF vs Regular View: Why use a TVF?

sql – Code Example

```
-- Regular view: no parameters  
CREATE VIEW electronics_products AS  
    SELECT * FROM products WHERE category_id = 1;  
-- Problem: one view per category, or no category filtering  
  
-- Table-valued function: accepts any category_id parameter  
SELECT * FROM get_products_by_category(1); -- Electronics  
SELECT * FROM get_products_by_category(2); -- Fitness  
SELECT * FROM get_products_by_category(5); -- Kitchen  
-- One function, parameterised – replaces N static views
```

ANALOGY

A regular view is like a fixed report template — same report every time. A table-valued function is like a report template with a parameter field — same logic, but you fill in the category/date/region before running it.

WARNING

In PostgreSQL, table-valued functions using **RETURN QUERY** re-execute the inner SELECT for every call. For heavy analytical queries called frequently, materialize the result into a temporary table instead:

sql – Code Example

```
-- Materialise for expensive TVFs called multiple times  
CREATE TEMP TABLE tmp_category_products AS  
    SELECT * FROM get_products_by_category(1);  
-- Use tmp_category_products in subsequent queries  
DROP TABLE tmp_category_products;
```

INTERVIEW CARD

Frequency: Medium

Question: What is a table-valued function and how is it different from a view?

Expected Answer: A table-valued function returns a result set and is used in the FROM clause like a table. Unlike a view, it accepts parameters — making it a

parameterised view. A view has fixed logic; a TVF can accept a category ID, date range, or any filter parameter and apply it dynamically. The tradeoff: a view can be indexed (materialized view), but a TVF re-executes on every call and cannot be directly indexed. TVFs are best suited for parameterised lookups where the result set changes based on input.

Follow-up: How does a TVF differ from a subquery in FROM? (Answer: Functionally similar — both appear in FROM and return a result set. The key difference is reusability: a TVF is named, stored in the database, and callable from multiple queries and applications. A subquery is inline, one-off, and must be re-written everywhere it is needed.)

🔗 PRACTICE QUIZ

1. In which SQL clause do you use a table-valued function?
2. What advantage does a TVF have over a regular view?
3. When should you materialise a TVF result into a temp table?
4. Write the RETURNS TABLE declaration for a TVF that returns product_id INT, name TEXT, and price NUMERIC.

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. The FROM clause — treated exactly like a table or subquery.
2. A TVF accepts parameters — it is a parameterised view. A regular view has fixed logic and returns the same columns every time with no input arguments.
3. When the TVF contains an expensive query that will be joined or filtered multiple times in the same session — materialising avoids re-executing the expensive query for each use.
4. `RETURNS TABLE(product_id INT, name TEXT, price NUMERIC)`

🔗 CHECKPOINT QUESTION

Write a table-valued function `get_customer_orders(p_customer_id INT, p_status TEXT)` that returns *orderid*, *orderdate*, *totalamount*, and a computed column called *dayssinceorder* for all orders matching both the customer and status. Use `CURRENTDATE - order_date` for the days calculation. Show how you would call it to find all pending orders for customer 101.

STORED PROCEDURES — ACTIONS, TRANSACTIONS, AND EXCEPTION HANDLING

ChatGPT • AI Assistant

A vertical flowchart showing the execution path of `process_daily_orders`. Start node (blue): 'CALL process_daily_orders(date)'. Four sequential action nodes connected by arrows: Node 1 (teal): 'Step 1 — SELECT: compute order counts and revenue into variables.' Node 2 (green): 'Step 2 — INSERT INTO order_summary_daily (upsert).' Node 3 (green): 'Step 3 — UPDATE orders SET status = expired.' Node 4 (green): 'Step 4 — INSERT INTO procedure_run_log SUCCESS.' Node 5 (gold): 'COMMIT — all writes committed atomically.' A dashed red error arrow branches from Node 3 downward to Exception path: Exception Node 1 (red): 'ROLLBACK — Step 2 and Step 3 undone.' Exception Node 2 (orange): 'INSERT into procedure_run_log FAILED (SQLERRM).' Exception Node 3 (orange): 'COMMIT — only the failure log committed.' Exception Node 4 (red): 'RAISE — exception propagated to caller (Airflow marks task failed).' A bracket on the left spans Steps 2-4 with label 'ONE transaction — all or nothing.' Clean flowchart with two paths: green success path and red exception path.

Stored Procedure Transaction Flow — Commit and Rollback Paths

sql – Stored Procedure – Daily Order Processing with Transaction and Exception Handling

```
-- Full stored procedure with transaction control
CREATE OR REPLACE PROCEDURE process_daily_orders(p_summary_date DATE)
LANGUAGE plpgsql
AS $$
DECLARE
    v_total_orders INT := 0;
    v_total_revenue NUMERIC := 0;
BEGIN
    -- Step 1: compute summary values into variables
    SELECT COUNT(*),
           COALESCE(SUM(total_amount), 0)
    INTO v_total_orders, v_total_revenue
    FROM orders
    WHERE order_date = p_summary_date
           AND status ≠ 'cancelled';

    -- Step 2: upsert into summary table
    INSERT INTO order_summary_daily(
        summary_date, total_orders, total_revenue, created_at
    )
    VALUES (p_summary_date, v_total_orders, v_total_revenue, NOW())
    ON CONFLICT (summary_date) DO UPDATE SET
        total_orders = EXCLUDED.total_orders,
        total_revenue = EXCLUDED.total_revenue,
        created_at = EXCLUDED.created_at;

    -- Step 3: expire old pending orders
    UPDATE orders
    SET status = 'expired'
```

```
WHERE order_date < p_summary_date - INTERVAL '30 days'
AND status = 'pending';

-- Step 4: log success
INSERT INTO procedure_run_log(procedure_name, run_date, status,
message)
VALUES ('process_daily_orders', NOW(), 'SUCCESS',
FORMAT('Orders: %s | Revenue: %.2f',
v_total_orders, v_total_revenue));

COMMIT; -- commit all three writes atomically

EXCEPTION WHEN OTHERS THEN
ROLLBACK; -- undo Steps 2 and 3
INSERT INTO procedure_run_log(procedure_name, run_date, status,
message)
VALUES ('process_daily_orders', NOW(), 'FAILED', SQLERRM);
COMMIT; -- commit only the failure log
RAISE; -- propagate exception to caller
END;
$$;

-- Call the procedure
CALL process_daily_orders(CURRENT_DATE - 1);
CALL process_daily_orders('2024-03-15'::DATE);

-- Verify results
SELECT * FROM order_summary_daily ORDER BY summary_date DESC LIMIT 5;
SELECT * FROM procedure_run_log ORDER BY run_date DESC LIMIT 10;
```

Student

Now stored procedures. You said they manage transactions and can do INSERT, UPDATE, DELETE. Show me a real example.

ChatGPT • AI Assistant

CONCEPT CARD

A **stored procedure** is a named, stored block of procedural SQL that performs actions — it modifies data, manages transactions, handles exceptions, and can run

complex multi-step workflows. You call it with **CALL** and it executes the full workflow atomically.

Key differences from functions:

- Can **COMMIT** and **ROLLBACK** inside the procedure body.
- Can contain **INSERT** , **UPDATE** , **DELETE** .
- Has full exception handling with **EXCEPTION** / **WHEN OTHERS THEN** .
- Cannot be embedded in a **SELECT** expression.

EXAMPLE CARD

Example 1 — Stored procedure: process daily orders summary

sql – Code Example

```
CREATE OR REPLACE PROCEDURE process_daily_orders(p_summary_date DATE)
LANGUAGE plpgsql
AS $$
DECLARE
    v_total_orders INT;
    v_total_revenue NUMERIC;
BEGIN
    -- Step 1: Compute today's order summary
    SELECT COUNT(*),
           COALESCE(SUM(total_amount), 0)
    INTO v_total_orders, v_total_revenue
    FROM orders
    WHERE order_date = p_summary_date
           AND status ≠ 'cancelled';

    -- Step 2: Insert into summary table (upsert – replace if exists)
    INSERT INTO order_summary_daily(
        summary_date, total_orders, total_revenue, created_at
    )
    VALUES (
        p_summary_date, v_total_orders, v_total_revenue, NOW()
    )
    ON CONFLICT (summary_date)
    DO UPDATE SET
        total_orders = EXCLUDED.total_orders,
        total_revenue = EXCLUDED.total_revenue,
        created_at = EXCLUDED.created_at;

    -- Step 3: Mark pending orders older than 30 days as expired
    UPDATE orders
    SET status = 'expired'
```

```
WHERE order_date < p_summary_date - INTERVAL '30 days'
AND status = 'pending';

-- Step 4: Log the procedure run
INSERT INTO procedure_run_log(procedure_name, run_date, status,
message)
VALUES ('process_daily_orders', NOW(), 'SUCCESS',
FORMAT('Processed %s orders, revenue %.2f',
v_total_orders, v_total_revenue));

COMMIT; -- commit all four steps together

EXCEPTION WHEN OTHERS THEN
-- If any step fails, roll back everything
ROLLBACK;
-- Log the failure
INSERT INTO procedure_run_log(procedure_name, run_date, status,
message)
VALUES ('process_daily_orders', NOW(), 'FAILED', SQLERRM);
COMMIT; -- commit the failure log even though the main work
rolled back
RAISE; -- re-raise the exception so the caller knows it failed
END;
$$;
```

Calling the procedure:

sql – Code Example

```
-- Run for yesterday
CALL process_daily_orders(CURRENT_DATE - 1);

-- In Airflow (PostgresOperator):
-- sql = "CALL process_daily_orders('{{ ds }}'::DATE);"
```

Example 2 — Understanding the transaction block:

sql – Code Example

```
-- All four steps inside the procedure execute as ONE transaction:
-- Step 1: SELECT (read only – no transaction concern)
-- Step 2: INSERT INTO order_summary_daily
-- Step 3: UPDATE orders SET status = 'expired'
-- Step 4: INSERT INTO procedure_run_log
-- COMMIT → all three writes committed together
```

```
-- If Step 3 fails:  
-- → ROLLBACK undoes Step 2 and Step 3  
-- → EXCEPTION block runs: logs failure and commits ONLY the failure  
log  
-- → The summary table and order status remain unchanged  
-- → Data is consistent
```

Student

This looks like exactly what you would put in an Airflow DAG. Is that right?

ChatGPT • AI Assistant

🌐 REAL WORLD APPLICATION

Exactly. The standard pattern in data engineering pipelines is:

python — Code Example

```
# Airflow DAG using PostgresOperator  
from airflow.providers.postgres.operators.postgres import  
PostgresOperator  
  
process_orders_task = PostgresOperator(  
    task_id='process_daily_orders',  
    postgres_conn_id='warehouse_db',  
    sql="CALL process_daily_orders('{{ ds }}'::DATE);"  
)
```

The Airflow task is a single **CALL** statement. All the complex logic — the selects, inserts, updates, error handling, logging — lives in the database procedure. The pipeline code stays clean.

👤 INTERVIEW CARD

Frequency: High

Question: What are the advantages of stored procedures for data engineering workflows?

Expected Answer: Three main advantages. First, logic lives close to the data — the procedure executes inside the database engine with no network round-trips between steps. Second, atomicity — all steps run in one transaction; either everything commits or everything rolls back, giving you consistent state on failure. Third,

pipeline simplification — the Airflow task or scheduler job is a single CALL statement; all complex SQL logic is encapsulated in the procedure and versioned separately from pipeline code. Limitations: procedures are stored in the database and are harder to version control with Git, harder to unit test in isolation, and create tight coupling between the transformation logic and the database platform.

Follow-up: What is the difference between COMMIT inside a stored procedure and COMMIT at the end of a transaction block? (Answer: In PostgreSQL, a procedure called with CALL can explicitly COMMIT mid-procedure — each COMMIT creates a savepoint. A function cannot COMMIT at all. Transaction control inside procedures allows multi-step workflows where some steps must commit before others begin.)

🔗 PRACTICE QUIZ

1. Can a stored procedure COMMIT inside its body?
2. What happens to all pending writes if the EXCEPTION block executes ROLLBACK?
3. Why is a stored procedure better than a Python script for atomic multi-table updates?
4. Predict: if Step 3 (UPDATE) in `process_daily_orders` fails, what is the state of Step 2 (INSERT into summary table)?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. Yes — in PostgreSQL, CALL allows explicit COMMIT and ROLLBACK inside a procedure body. Functions cannot.
2. All uncommitted writes from the current transaction block are undone — the database returns to the state it was in before the procedure started (or before the last COMMIT checkpoint).
3. A stored procedure executes atomically inside the database — all steps are one transaction. A Python script with multiple database round-trips cannot guarantee atomicity without explicit transaction management, and network failures between steps can leave data in a partial state.
4. With ROLLBACK in the EXCEPTION block, Step 2 is also rolled back — the INSERT into `ordersummarydaily` is undone. The summary table remains as it was before the procedure ran.

🔗 CHECKPOINT QUESTION

You need to write a stored procedure `archive_old_orders(p_cutoff_date DATE)` that: (1) copies all cancelled orders older than `p_cutoffdate` into an `orders_archive` table, (2) deletes those rows from the active `orders` table, and (3) logs the count of archived rows to an `archive_log` table. Write the procedure with proper DECLARE, transaction handling, and EXCEPTION block. What ON DELETE cascade consideration should you check before step 2?

SNOWFLAKE STORED PROCEDURES — JAVASCRIPT AND PYTHON HANDLERS

ChatGPT • AI Assistant

A three-column comparison diagram showing the three Snowflake procedure styles. Column 1 (orange background, label 'JavaScript Handler'): shows LANGUAGE JAVASCRIPT with a code snippet using snowflake.createStatement() and rs.getColumnValue(). Badges: 'Legacy — widely used in existing code. Verbose. No native SQL syntax. Column access by name string.' Column 2 (blue background, label 'Snowflake Scripting'): shows LANGUAGE SQL with DECLARE, BEGIN, IF/ELSEIF, FOR cursor loop, EXCEPTION syntax. Badges: 'Modern — SQL-native. Familiar syntax. Recommended for new procedures. DECLARE, IF, FOR, EXCEPTION blocks.' Column 3 (green background, label 'Python Handler'): shows LANGUAGE PYTHON with handler function using session.sql(). Badges: 'Modern — full Python ecosystem. Snowpark DataFrames. Best for complex logic and ML workflows.' Below all three columns, a recommendation strip reads: 'For SQL engineers: use Snowflake Scripting. For Python engineers: use Python handler. For legacy maintenance: JavaScript.' Clean three-column layout, color-coded, code snippets in monospace.

Snowflake Stored Procedure Handler Comparison

sql — Snowflake Procedures — JavaScript Handler, Scripting, and Cursor Loop

```
-- =====
-- SNOWFLAKE JAVASCRIPT HANDLER
-- =====

CREATE OR REPLACE PROCEDURE process_daily_sf(summary_date DATE)
RETURNS STRING
LANGUAGE JAVASCRIPT
AS
$$
    try {
        var stmt = snowflake.createStatement({
            sqlText: 'SELECT COUNT(*) AS cnt FROM orders WHERE
order_date = :1',
            binds: [SUMMARY_DATE]
        });
        var rs = stmt.execute();
        rs.next();
        var cnt = rs.getColumnValue('CNT');
        return 'Orders: ' + cnt;
    } catch(err) {
        throw err;
    }
```

```
}
$$;

-- =====
-- SNOWFLAKE SCRIPTING (SQL-native, recommended)
-- =====

CREATE OR REPLACE PROCEDURE process_daily_scripting(summary_date DATE)
RETURNS STRING
LANGUAGE SQL
AS
$$
DECLARE
    v_orders  INTEGER DEFAULT 0;
    v_revenue FLOAT   DEFAULT 0;
BEGIN
    SELECT COUNT(*), COALESCE(SUM(total_amount), 0)
    INTO :v_orders, :v_revenue
    FROM orders
    WHERE order_date = :summary_date AND status ≠ 'cancelled';

    IF (v_orders = 0) THEN
        RETURN 'No orders for ' || summary_date::STRING;
    END IF;

    MERGE INTO order_summary_daily tgt
    USING (SELECT :summary_date AS d, :v_orders AS o, :v_revenue AS r)
src
    ON tgt.summary_date = src.d
    WHEN MATCHED THEN UPDATE SET total_orders=src.o,
total_revenue=src.r
    WHEN NOT MATCHED THEN INSERT
(summary_date,total_orders,total_revenue)
VALUES (src.d, src.o, src.r);

    RETURN 'Done: ' || v_orders || ' orders';
EXCEPTION
    WHEN OTHER THEN
        LET msg := SQLERRM;
        INSERT INTO procedure_run_log VALUES
            ('process_daily', CURRENT_TIMESTAMP(), 'FAILED', :msg);
        RAISE;
END;
$$;

CALL process_daily_scripting('2024-03-15'::DATE);
```

```

-- =====
-- CURSOR LOOP (slow) vs SET-BASED (fast) comparison
-- =====

-- SLOW: cursor loop – 500K individual UPDATE calls
CREATE OR REPLACE PROCEDURE update_tiers_cursor()
RETURNS STRING LANGUAGE SQL AS
$$
DECLARE
    c_cust CURSOR FOR SELECT customer_id, COALESCE(SUM(total_amount),0)
AS spend
                        FROM orders WHERE status='delivered' GROUP BY
customer_id;
    v_count INTEGER DEFAULT 0;
BEGIN
    FOR rec IN c_cust DO
        UPDATE customers SET tier =
            CASE WHEN rec.spend ≥ 50000 THEN 'Gold'
                 WHEN rec.spend ≥ 10000 THEN 'Silver' ELSE 'Bronze'
            END
        WHERE customer_id = rec.customer_id;
        v_count := v_count + 1;
    END FOR;
    RETURN 'Updated: ' || v_count;
END;
$$;

-- FAST: set-based – one SQL statement
UPDATE customers c
SET    tier = CASE
    WHEN spend.lifetime_spend ≥ 50000 THEN 'Gold'
    WHEN spend.lifetime_spend ≥ 10000 THEN 'Silver'
    ELSE 'Bronze' END
FROM (
    SELECT customer_id, COALESCE(SUM(total_amount), 0) AS
lifetime_spend
    FROM    orders WHERE status = 'delivered' GROUP BY customer_id
) spend
WHERE c.customer_id = spend.customer_id;

```

Student

This programme uses Snowflake. How are stored procedures different there? I heard they use JavaScript or Python instead of SQL.

CONCEPT CARD

Snowflake stored procedures use a **handler language** — the procedure body is written in JavaScript, Python, Java, Scala, or Snowflake Scripting (SQL-based procedural language). The most commonly used in practice are **JavaScript** (legacy, widely used) and **Python** (modern, recommended for new development).

Key Snowflake specifics:

- Procedures are called with `CALL procedure_name();` — same as PostgreSQL.
- JavaScript handler: use the `snowflake` object to execute SQL statements.
- Python handler: use `snowflake.snowpark` session to run DataFrames or SQL.
- Snowflake Scripting: SQL-native procedural syntax — closest to PL/pgSQL.
- Snowflake does NOT support `COMMIT` / `ROLLBACK` inside JavaScript procedures by default — use Snowflake Scripting or set `EXECUTE AS CALLER` with explicit transaction control.

EXAMPLE CARD**Example 1 — Snowflake JavaScript handler: process daily orders**

sql — Code Example

```
CREATE OR REPLACE PROCEDURE process_daily_orders_sf(summary_date DATE)
RETURNS STRING
LANGUAGE JAVASCRIPT
AS
$$
    try {
        // Step 1: Compute summary
        var stmt1 = snowflake.createStatement({
            sqlText: `
                SELECT COUNT(*) AS cnt,
                       COALESCE(SUM(total_amount), 0) AS rev
                FROM   orders
                WHERE  order_date = :1
                       AND status ≠ 'cancelled`,
            binds: [SUMMARY_DATE]
        });
        var rs1 = stmt1.execute();
        rs1.next();
        var totalOrders  = rs1.getColumnValue('CNT');
        var totalRevenue = rs1.getColumnValue('REV');
```

```
// Step 2: Insert into summary table
snowflake.execute({
  sqlText: `
    MERGE INTO order_summary_daily tgt
    USING (SELECT :1 AS summary_date,
                :2 AS total_orders,
                :3 AS total_revenue) src
    ON tgt.summary_date = src.summary_date
    WHEN MATCHED THEN UPDATE SET
      total_orders = src.total_orders,
      total_revenue = src.total_revenue
    WHEN NOT MATCHED THEN INSERT VALUES
      (src.summary_date, src.total_orders,
src.total_revenue, CURRENT_TIMESTAMP())`,
  binds: [SUMMARY_DATE, totalOrders, totalRevenue]
});

return 'SUCCESS: ' + totalOrders + ' orders, revenue ' +
totalRevenue;

} catch(err) {
  // Log failure
  snowflake.execute({
    sqlText: `INSERT INTO procedure_run_log VALUES
      ('process_daily_orders_sf', CURRENT_TIMESTAMP(),
'FAILED', :1)`,
    binds: [err.message]
  });
  throw err; // re-raise so Snowflake marks the call as failed
}
$$;
```

Example 2 — Snowflake Scripting (SQL-native, recommended for new procedures):

sql – Code Example

```
CREATE OR REPLACE PROCEDURE
process_daily_orders_scripting(summary_date DATE)
RETURNS STRING
LANGUAGE SQL
AS
$$
DECLARE
```

```
v_total_orders  INTEGER DEFAULT 0;
v_total_revenue FLOAT   DEFAULT 0;
v_message       STRING;
BEGIN
    -- Variable assignment from SELECT
    SELECT COUNT(*), COALESCE(SUM(total_amount), 0)
    INTO :v_total_orders, :v_total_revenue
    FROM orders
    WHERE order_date = :summary_date
        AND status ≠ 'cancelled';

    -- Conditional logic
    IF (v_total_orders = 0) THEN
        v_message := 'No orders found for ' || summary_date::STRING;
        RETURN v_message;
    END IF;

    -- Merge into summary table
    MERGE INTO order_summary_daily tgt
    USING (SELECT :summary_date      AS summary_date,
                  :v_total_orders    AS total_orders,
                  :v_total_revenue   AS total_revenue) src
    ON tgt.summary_date = src.summary_date
    WHEN MATCHED THEN UPDATE SET
        total_orders = src.total_orders,
        total_revenue = src.total_revenue
    WHEN NOT MATCHED THEN INSERT
        (summary_date, total_orders, total_revenue, created_at)
    VALUES
        (src.summary_date, src.total_orders, src.total_revenue,
        CURRENT_TIMESTAMP());

    v_message := 'Processed ' || v_total_orders || ' orders, revenue:
' || v_total_revenue;
    RETURN v_message;

EXCEPTION
    WHEN OTHER THEN
        LET err_msg := SQLERRM;
        INSERT INTO procedure_run_log
        VALUES ('process_daily_orders', CURRENT_TIMESTAMP(), 'FAILED',
:err_msg);
        RAISE;
END;
$;
```

```
-- Call from SQL
CALL process_daily_orders_scripting('2024-03-15'::DATE);
```

Example 3 — Snowflake Scripting: cursor loop over a result set

sql – Code Example

```
CREATE OR REPLACE PROCEDURE update_customer_tiers()
RETURNS STRING
LANGUAGE SQL
AS
$$
DECLARE
    v_customer_id  INTEGER;
    v_lifetime_spend FLOAT;
    v_tier          STRING;
    v_count         INTEGER DEFAULT 0;

    -- Declare a cursor over customer lifetime spend
    c_customers CURSOR FOR
        SELECT c.customer_id,
               COALESCE(SUM(o.total_amount), 0) AS lifetime_spend
        FROM   customers c
        LEFT JOIN orders o ON c.customer_id = o.customer_id
           AND o.status = 'delivered'
        GROUP BY c.customer_id;
BEGIN
    -- Loop through every customer
    FOR customer_rec IN c_customers DO
        v_customer_id := customer_rec.customer_id;
        v_lifetime_spend := customer_rec.lifetime_spend;

        -- Determine tier
        IF (v_lifetime_spend ≥ 50000) THEN
            v_tier := 'Gold';
        ELSEIF (v_lifetime_spend ≥ 10000) THEN
            v_tier := 'Silver';
        ELSE
            v_tier := 'Bronze';
        END IF;

        -- Update customer tier
        UPDATE customers
        SET    tier = :v_tier
        WHERE  customer_id = :v_customer_id
           AND tier ≠ :v_tier; -- only update if tier changed
    END LOOP;
END;
```

```
        v_count := v_count + 1;
    END FOR;

    RETURN 'Updated tiers for ' || v_count || ' customers';
END;
$$;

CALL update_customer_tiers();
```

⚠ WARNING

Cursor loops in Snowflake (and PostgreSQL) are slow on large tables. A cursor loop processes rows one by one — N UPDATE statements for N customers. For 500,000 customers, that is 500,000 individual UPDATE calls. The set-based equivalent is thousands of times faster:

sql — Code Example

```
-- SET-BASED EQUIVALENT (vastly faster than cursor loop):
UPDATE customers c
SET tier = CASE
    WHEN spend.lifetime_spend ≥ 50000 THEN 'Gold'
    WHEN spend.lifetime_spend ≥ 10000 THEN 'Silver'
    ELSE 'Bronze'
END
FROM (
    SELECT customer_id, COALESCE(SUM(total_amount), 0) AS
lifetime_spend
    FROM orders WHERE status = 'delivered'
    GROUP BY customer_id
) spend
WHERE c.customer_id = spend.customer_id;
-- One SQL statement instead of 500,000 UPDATE calls
```

🧠 INTERVIEW CARD

Frequency: High (specifically for Snowflake roles)

Question: How do Snowflake stored procedures differ from PostgreSQL stored procedures?

Expected Answer: Snowflake stored procedures use handler languages — JavaScript, Python, Java, Scala, or Snowflake Scripting (SQL-based). The most common are JavaScript (legacy, widely used) and Snowflake Scripting (modern, SQL-native, recommended). Unlike PostgreSQL PL/pgSQL, JavaScript handlers use

the snowflake object to create and execute statements, and column values are accessed by name using getColumnValue(). Snowflake Scripting uses familiar SQL syntax with DECLARE, BEGIN/END, IF/ELSEIF, cursor FOR loops, and EXCEPTION handling. Both are called with CALL. Key Snowflake-specific: procedures return a single value (STRING/VARIANT), variables use colon prefix (:var_name) in SQL statements, and cursors are looped with FOR rec IN cursor DO syntax.

Follow-up: Why should you avoid cursor loops in Snowflake procedures for large tables? (Answer: Cursor loops process rows one at a time — N rows = N individual DML statements. Snowflake is optimized for set-based operations that process millions of rows in parallel. A single UPDATE...FROM with a grouped subquery replaces 500,000 individual UPDATE calls and runs in seconds instead of hours.)

🔗 PRACTICE QUIZ

1. In a Snowflake JavaScript procedure, what object do you use to execute SQL statements?
2. What is the Snowflake Scripting syntax for variable assignment from a SELECT result?
3. Why should you use set-based SQL instead of cursor loops for large table updates?
4. What does EXECUTE AS CALLER mean in a Snowflake procedure?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔗 ANSWER KEY

1. The `snowflake` object — e.g., `snowflake.execute({sqlText: ' ... '})` or `snowflake.createStatement({sqlText: ' ... '}).execute()`.
2. `SELECT col INTO :variable_name FROM table WHERE condition;` — colon prefix on the variable name.
3. Cursor loops execute one DML statement per row — O(N) individual operations. Snowflake's MPP engine processes set-based SQL across all nodes in parallel — one statement handles millions of rows. The speedup is typically 100x to 10,000x for large tables.
4. EXECUTE AS CALLER means the procedure runs with the privileges of the calling user rather than the procedure owner. Used when the procedure should respect the caller's access controls rather than the owner's elevated permissions.

🔗 CHECKPOINT QUESTION

Rewrite the Snowflake cursor loop procedure `update_customer_tiers()` as a set-based single SQL statement inside a Snowflake Scripting procedure. Explain why this is faster on a table with 500,000 customers and how you would verify the correctness of both approaches.

LIMITATIONS AND WHEN TO USE PROCEDURES VS DBT VS PYTHON

ChatGPT • AI Assistant

A three-column decision matrix diagram. Column 1 (orange background, header 'STORED PROCEDURE'): lists use cases with green checkmarks: SCD Type 2 MERGE, atomic multi-table writes, operational bulk updates (expire orders, deactivate users), audit logging co-located with the write, Airflow CALL as single task. Lists limitations with red X marks: hard to Git version, no unit testing, platform-specific language. Column 2 (blue background, header 'dbt MODEL'): lists use cases with green checkmarks: all analytical SELECT transformations, fact and dimension table builds, incremental processing, anything needing schema tests and documentation. Lists limitations with red X: cannot manage transactions, no procedural logic, SELECT-only. Column 3 (green background, header 'PYTHON (Airflow / dbt Python)'): lists use cases with green checkmarks: ML feature engineering, cross-system workflows (S3 to Snowflake), complex string/API logic, anything needing pytest unit tests. Lists limitations with red X: slower than SQL for set-based operations, more infrastructure to manage. Below the three columns, a rule strip reads: 'Default to dbt. Add stored procedures for transactional operational workflows. Use Python for logic SQL cannot express.' Clean three-column layout, checkmark/X icons, consistent color scheme.

Decision Matrix — Stored Procedure vs dbt vs Python

sql – Limitations in Practice – Version Control Pattern and dbt vs Procedure Decision

```
-- =====
-- VERSION CONTROL PATTERN for stored procedures
-- =====

-- File: migrations/V012__update_process_daily_orders.sql
-- Committed to Git, deployed via Flyway/Liquibase

CREATE OR REPLACE PROCEDURE process_daily_orders(p_date DATE)
RETURNS STRING
LANGUAGE SQL
AS
$$
DECLARE
    v_count INTEGER DEFAULT 0;
BEGIN
    -- ... procedure body ...
    RETURN 'OK: ' || v_count;
END;
$$;

-- ↑ This file is the source of truth.
-- Never edit in Snowflake UI without updating this file.
```

```

-- =====
-- dbt equivalent for analytical transforms (no procedure needed)
-- =====
-- models/customer_metrics.sql (dbt model)
-- {{ config(materialized='incremental', unique_key='customer_id') }}
SELECT
    c.customer_id,
    c.name,
    c.city,
    COALESCE(SUM(o.total_amount), 0)           AS lifetime_spend,
    COUNT(o.order_id)                         AS total_orders,
    CASE
        WHEN COALESCE(SUM(o.total_amount),0) ≥ 50000 THEN 'Gold'
        WHEN COALESCE(SUM(o.total_amount),0) ≥ 10000 THEN 'Silver'
        ELSE 'Bronze'
    END                                       AS tier,
    MAX(o.order_date)                       AS last_order_date
FROM customers c
LEFT JOIN orders o ON c.customer_id = o.customer_id
    AND o.status = 'delivered'
GROUP BY c.customer_id, c.name, c.city;
-- ↑ Versioned, testable, documented – dbt schema test:
-- models/schema.yml:
-- - name: customer_metrics
--   columns:
--     - name: customer_id
--       tests: [unique, not_null]
--     - name: tier
--       tests:
--         - accepted_values: {values: [Gold, Silver, Bronze]}

-- =====
-- OPERATIONAL WORKFLOW that DOES need a procedure:
-- Deactivate customers and log each – atomic transaction
-- =====
-- (preview of Practice Problem 2)
CREATE OR REPLACE PROCEDURE deactivate_inactive_customers()
RETURNS STRING LANGUAGE SQL AS
$$
DECLARE
    v_count INTEGER DEFAULT 0;
    c_inactive CURSOR FOR
        SELECT c.customer_id, c.name
        FROM customers c
        LEFT JOIN orders o ON c.customer_id = o.customer_id
            AND o.order_date ≥ CURRENT_DATE - 180

```

```
WHERE c.is_active = TRUE
GROUP BY c.customer_id, c.name
HAVING COUNT(o.order_id) = 0;

BEGIN
-- ... loop + UPDATE + INSERT audit log ...
RETURN 'Deactivated: ' || v_count;
END;
$$;
```

Student

Are stored procedures always the right tool? You said there are limitations.

ChatGPT • AI Assistant

CONCEPT CARD

Stored procedures are powerful but they come with real trade-offs. In modern data engineering, they are used selectively — not as the default for all transformations.

The three honest limitations:**1. Hard to version control**

sql – Code Example

```
-- The procedure lives INSIDE the database.
-- To version it with Git, you must:
--   a) maintain a .sql file in your repository, AND
--   b) deploy it to the database on every change
-- If someone edits it directly in Snowflake UI or psql,
-- the repo and the database are out of sync.
-- There is no automatic Git commit when you run CREATE OR REPLACE
PROCEDURE.
```

2. Hard to test in isolation

sql – Code Example

```
-- You cannot unit test a stored procedure like a Python function.
-- Testing requires:
--   a) a live database connection
--   b) test data already loaded
```

```
-- c) cleaning up state after each test run
-- There is no equivalent of pytest + mock for stored procedures.
```

3. Tight database coupling

sql – Code Example

```
-- Logic written in PL/pgSQL runs only on PostgreSQL.
-- Logic written in Snowflake Scripting runs only on Snowflake.
-- If you migrate from PostgreSQL to Snowflake,
-- every stored procedure must be rewritten.
-- Python logic in application code or dbt models is portable.
```

CONCEPT CARD

When to use each approach:

Use stored procedures for:

- **SCD Type 2 MERGE operations** — complex row-level change tracking that is awkward in dbt
- **Audit logging workflows** — procedures that log changes atomically with the change itself
- **Bulk status updates** — nightly jobs that update order statuses, customer tiers, expiry flags
- **Operational workflows called from schedulers** — the CALL statement is the cleanest interface for Airflow
- **Cross-table transactions** — multi-table updates that must be atomic

Use dbt models for:

- **All analytical transformations** — SELECT-based logic that builds facts and dimensions
- **Anything that needs testing** — dbt tests, schema tests, source freshness checks
- **Anything that needs documentation** — dbt generates a data catalogue automatically
- **Incremental models** — dbt incremental materializations are more maintainable than custom procedures

Use Python (Airflow, dbt Python models) for:

- **Complex logic that SQL cannot express cleanly** — ML feature engineering, string manipulation, API calls
- **Cross-system workflows** — read from S3, transform, write to Snowflake
- **Testing** — pytest, unit tests, mock database connections

EXAMPLE CARD**Decision matrix in practice:**

code – Code Example

Transformation need	→ Tool
Build a fact table from orders	→ dbt model (SELECT)
SCD Type 2 customer dimension MERGE	→ dbt snapshot OR stored procedure
Nightly expired order cleanup	→ Stored procedure (UPDATE + log)
Customer tier classification	→ dbt model + SQL CASE expression
Bulk deactivate inactive users	→ Stored procedure (multi-step + audit log)
ML feature engineering	→ Python (dbt Python model or Airflow task)
Load from S3 to Snowflake stage	→ Snowflake COPY command + Airflow task

Student

So procedures are not going away — they just have a specific place?

ChatGPT • AI Assistant

Exactly. The pragmatic position is:

- Default to **dbt** for transformations — it is versioned, tested, documented.
- Use **stored procedures** for operational workflows where atomicity and close-to-data execution matter.
- Use **Python** for anything that needs programmatic logic, external API calls, or portability.
- Never use stored procedures to work around the absence of dbt or to avoid learning SQL properly.

🔗 INTERVIEW CARD

Frequency: High (data engineering roles)

Question: When would you use a stored procedure vs a dbt model for a data transformation?

Expected Answer: dbt models are the default for analytical transformations — they are version-controlled in Git, testable, documented, and run as SELECT statements.

Use stored procedures when: the workflow requires atomic multi-table writes inside a single transaction; you need to log changes atomically with the operation itself (e.g., SCD Type 2 with audit log); the workflow involves complex row-by-row operational logic like bulk deactivation; or you need to call it cleanly from Airflow with a single CALL statement. Key limitations of stored procedures: stored in the database (not Git natively), hard to unit test in isolation, and tightly coupled to the database platform — PL/pgSQL will not run on Snowflake.

Follow-up: How do you version control a stored procedure in a Git-based workflow? (Answer: Maintain the CREATE OR REPLACE PROCEDURE statement in a .sql file in your repository. Deploy via a migration tool (Flyway, Liquibase, dbt run-operation) as part of your CI/CD pipeline. Never edit procedures directly in the database UI without also updating the repository file.)

🔍 PRACTICE QUIZ

1. Name two data engineering use cases where a stored procedure is clearly the right tool over dbt.
2. Why is a stored procedure harder to test than a Python function?
3. What is the portability problem with stored procedures?
4. In a dbt vs stored procedure decision: a nightly job reads from three tables, computes customer lifetime value, and writes to a customer_metrics table. Which tool would you use and why?

▼ Reveal Correct Answer

Answer Checklist: Check calculations, verify logic, check edge cases.

🔑 ANSWER KEY

1. Any two of: SCD Type 2 MERGE with audit log, bulk status updates (expire old orders), atomic deactivation workflows with logging, operational MERGE operations that must be transactional.
2. A stored procedure requires a live database connection, pre-loaded test data, and cleanup after each test run. There is no equivalent of pytest + mock — you cannot inject dependencies or run in isolation. A Python function can be tested with mocked inputs and no database at all.
3. Stored procedures are written in database-specific procedural languages — PL/pgSQL for PostgreSQL, Snowflake Scripting for Snowflake, T-SQL for SQL Server. Migrating between platforms requires rewriting every procedure. Python logic and dbt SQL are more portable.

4. dbt model — this is a SELECT-based analytical transformation with no transaction management needed. dbt provides version control, schema tests (assert output is not null, unique), documentation, and incremental materialisation. A stored procedure would add complexity with no benefit for a pure SELECT-and-insert transformation.

🔍 CHECKPOINT QUESTION

Your team has a stored procedure `refresh_customer_metrics()` that runs nightly. A new engineer edited it directly in the Snowflake worksheet without updating the Git repository. Describe the operational problem this creates and the process you would put in place to prevent it from happening again.

PRACTICE PROBLEMS — SCALAR FUNCTION AND DEACTIVATION PROCEDURE

ChatGPT • AI Assistant

A vertical three-step workflow diagram with transaction bracket. Step 1 node (teal): 'INSERT INTO customer_deactivation_log — audit row per inactive customer. Uses NOT EXISTS subquery to identify customers with no recent orders.' Step 2 node (green): 'UPDATE customers SET is_active = FALSE — same NOT EXISTS filter. GET DIAGNOSTICS captures row count.' Step 3 node (blue): 'INSERT INTO procedure_run_log — logs deactivated count and timestamp.' A large curly bracket on the left spans all three steps with label 'ONE transaction — COMMIT at end.' An exception path branches from Step 2: red box 'ROLLBACK — all three steps undone.' Orange box 'INSERT failure log — committed separately.' Red box 'RAISE — propagated to Airflow.' Below the diagram, a verification query strip shows: 'SELECT * FROM customer_deactivation_log ORDER BY deactivated_at DESC LIMIT 20;' and 'SELECT COUNT(*) FROM customers WHERE is_active = FALSE;'. Clean three-step layout, transaction bracket on left, exception path on right.

Deactivation Procedure — Atomic Three-Step Workflow

sql — Practice Problems — Discount Function and Deactivation Procedure
Complete Solutions

```
-- =====
-- PROBLEM 1: Scalar discount function
-- =====

CREATE OR REPLACE FUNCTION get_discount_percent(
    order_total    NUMERIC,
    customer_tier  TEXT
)
RETURNS NUMERIC LANGUAGE plpgsql AS $$
DECLARE v_discount NUMERIC := 0;
BEGIN
    IF order_total IS NULL OR order_total < 500 THEN RETURN 0; END IF;
    CASE customer_tier
        WHEN 'Gold' THEN
            v_discount := 10;
```

```

        IF order_total > 5000 THEN v_discount := 15; END IF;
    WHEN 'Silver' THEN
        v_discount := 5;
        IF order_total > 3000 THEN v_discount := 8; END IF;
    WHEN 'Bronze' THEN
        IF order_total > 10000 THEN v_discount := 2; END IF;
    ELSE v_discount := 0;
END CASE;
RETURN v_discount;
END;
$$;

-- Test cases
SELECT get_discount_percent(6000, 'Gold');    -- 15
SELECT get_discount_percent(4000, 'Gold');    -- 10
SELECT get_discount_percent(3500, 'Silver');  -- 8
SELECT get_discount_percent(300, 'Gold');     -- 0 (< 500)

-- Apply in query
SELECT o.order_id, o.total_amount, c.tier,
       get_discount_percent(o.total_amount, c.tier) AS discount_pct,
       o.total_amount * (1 - get_discount_percent(o.total_amount,
c.tier)/100.0) AS final_price
FROM orders o JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status = 'pending' ORDER BY discount_pct DESC;

-- =====
-- PROBLEM 2: Deactivation procedure (PostgreSQL)
-- =====

CREATE OR REPLACE PROCEDURE deactivate_inactive_customers()
LANGUAGE plpgsql AS $$
DECLARE
    v_count    INT := 0;
    v_cutoff   DATE := CURRENT_DATE - INTERVAL '180 days';
BEGIN
    INSERT INTO customer_deactivation_log(customer_id, deactivated_at,
reason)
    SELECT c.customer_id, NOW(),
           'No orders since ' || v_cutoff::TEXT
    FROM   customers c
    WHERE  c.is_active = TRUE
           AND NOT EXISTS (
                SELECT 1 FROM orders o
                WHERE  o.customer_id = c.customer_id
                       AND o.order_date ≥ v_cutoff);
    GET DIAGNOSTICS v_count = ROW_COUNT;

```

```
UPDATE customers SET is_active = FALSE
WHERE is_active = TRUE
  AND NOT EXISTS (
    SELECT 1 FROM orders o
    WHERE o.customer_id = customers.customer_id
      AND o.order_date ≥ v_cutoff);

INSERT INTO procedure_run_log(procedure_name, run_date, status,
message)
VALUES ('deactivate_inactive_customers', NOW(), 'SUCCESS',
      FORMAT('Deactivated %s customers inactive since %s',
v_count, v_cutoff));
COMMIT;

EXCEPTION WHEN OTHERS THEN
  ROLLBACK;
  INSERT INTO procedure_run_log(procedure_name, run_date, status,
message)
VALUES ('deactivate_inactive_customers', NOW(), 'FAILED', SQLERRM);
COMMIT;
RAISE;

END;
$$;

CALL deactivate_inactive_customers();
SELECT * FROM customer_deactivation_log ORDER BY deactivated_at DESC
LIMIT 10;
SELECT * FROM procedure_run_log ORDER BY run_date DESC LIMIT 5;
```

Student

Can we now work through the two practice problems?

ChatGPT • AI Assistant

CONCEPT CARD

These two problems cover the most common interview tasks for stored procedures and functions in data engineering roles.

Practice Problem 1 — Scalar function: discount percentage by order total and customer tier

Requirements:

- Input: `order_total` NUMERIC , `customer_tier` TEXT
- Output: discount percentage as NUMERIC
- Rules:
- Gold tier: base 10% for any order + extra 5% if order > 5000 = max 15%
- Silver tier: base 5% for any order + extra 3% if order > 3000 = max 8%
- Bronze tier: 0% unless order > 10000, then 2%
- Orders below 500 never receive any discount regardless of tier

Solution:

sql – Code Example

```
CREATE OR REPLACE FUNCTION get_discount_percent(  
    order_total    NUMERIC,  
    customer_tier  TEXT  
)  
RETURNS NUMERIC  
LANGUAGE plpgsql  
AS $$  
DECLARE  
    v_discount NUMERIC := 0;  
BEGIN  
    -- No discount for small orders regardless of tier  
    IF order_total IS NULL OR order_total < 500 THEN  
        RETURN 0;  
    END IF;  
  
    -- Tier-based discount logic  
    CASE customer_tier  
        WHEN 'Gold' THEN  
            v_discount := 10; -- base Gold discount  
            IF order_total > 5000 THEN  
                v_discount := v_discount + 5; -- large order bonus =  
15%  
            END IF;  
  
        WHEN 'Silver' THEN  
            v_discount := 5; -- base Silver discount  
            IF order_total > 3000 THEN  
                v_discount := v_discount + 3; -- large order bonus =  
8%  
            END IF;
```

```

        WHEN 'Bronze' THEN
            IF order_total > 10000 THEN
                v_discount := 2; -- large order Bronze bonus
            ELSE
                v_discount := 0;
            END IF;

        ELSE
            v_discount := 0; -- unknown tier = no discount
        END CASE;

        RETURN v_discount;
    END;
$$;

-- Test the function:
SELECT get_discount_percent(6000, 'Gold');      -- should return 15
SELECT get_discount_percent(4000, 'Gold');      -- should return 10
SELECT get_discount_percent(3500, 'Silver');    -- should return 8
SELECT get_discount_percent(2000, 'Silver');    -- should return 5
SELECT get_discount_percent(11000, 'Bronze');   -- should return 2
SELECT get_discount_percent(5000, 'Bronze');    -- should return 0
SELECT get_discount_percent(300, 'Gold');        -- should return 0 (<
500)
SELECT get_discount_percent(NULL, 'Gold');       -- should return 0 (NULL
check)

-- Use in a pricing query:
SELECT o.order_id,
       o.total_amount,
       c.tier,
       get_discount_percent(o.total_amount, c.tier) AS discount_pct,
       o.total_amount * (1 - get_discount_percent(o.total_amount,
c.tier) / 100.0)
                                           AS final_price
FROM orders o
JOIN customers c ON o.customer_id = c.customer_id
WHERE o.status = 'pending'
ORDER BY discount_pct DESC, o.total_amount DESC;

```

Practice Problem 2 — Stored procedure: deactivate inactive customers with audit log

Requirements:

- Find all active customers who have had NO orders in the last 180 days

- Set `is_active = FALSE` on each such customer
- Insert one row per deactivated customer into `customer_deactivation_log`
- Log the total count and run status to `procedure_run_log`
- All writes must be atomic — if anything fails, roll back everything
- Return a summary string showing how many customers were deactivated

Solution:

sql – Code Example

```
-- Required tables (create if not existing):
-- customers: customer_id, name, email, is_active (BOOLEAN)
-- customer_deactivation_log:
--   log_id SERIAL PK, customer_id INT, deactivated_at TIMESTAMP,
--   reason TEXT
-- procedure_run_log:
--   log_id SERIAL PK, procedure_name TEXT, run_date TIMESTAMP,
--   status TEXT, message TEXT

CREATE OR REPLACE PROCEDURE deactivate_inactive_customers()
LANGUAGE plpgsql
AS $$
DECLARE
    v_deactivated_count INT := 0;
    v_cutoff_date       DATE := CURRENT_DATE - INTERVAL '180 days';
BEGIN
    -- Step 1: Insert one audit row per inactive customer BEFORE
    -- deactivating
    -- (so we have a record of who was deactivated and why)
    INSERT INTO customer_deactivation_log(
        customer_id, deactivated_at, reason
    )
    SELECT c.customer_id,
           NOW(),
           'No orders in last 180 days (last order before ' ||
           || v_cutoff_date::TEXT || ' )'
    FROM   customers c
    WHERE  c.is_active = TRUE
           AND NOT EXISTS (
               SELECT 1
               FROM    orders o
               WHERE   o.customer_id = c.customer_id
                       AND o.order_date ≥ v_cutoff_date
           );
```

```
-- Capture how many customers we are about to deactivate
GET DIAGNOSTICS v_deactivated_count = ROW_COUNT;

-- Step 2: Deactivate those customers (set-based update)
UPDATE customers c
SET    is_active = FALSE
WHERE  c.is_active = TRUE
      AND NOT EXISTS (
          SELECT 1
          FROM    orders o
          WHERE   o.customer_id = c.customer_id
                AND o.order_date ≥ v_cutoff_date
      );

-- Step 3: Log the procedure run result
INSERT INTO procedure_run_log(
    procedure_name, run_date, status, message
)
VALUES (
    'deactivate_inactive_customers',
    NOW(),
    'SUCCESS',
    FORMAT('Deactivated %s customers inactive since %s',
        v_deactivated_count, v_cutoff_date)
);

COMMIT; -- commit all three steps atomically

RAISE NOTICE 'Deactivated % customers', v_deactivated_count;

EXCEPTION WHEN OTHERS THEN
    ROLLBACK; -- undo all three steps
    INSERT INTO procedure_run_log(
        procedure_name, run_date, status, message
    )
    VALUES (
        'deactivate_inactive_customers',
        NOW(),
        'FAILED',
        SQLERRM
    );
    COMMIT; -- commit only the failure log
    RAISE; -- propagate exception
END;
$$;
```

```
-- Call the procedure
CALL deactivate_inactive_customers();

-- Verify results
SELECT * FROM customer_deactivation_log
ORDER BY deactivated_at DESC LIMIT 20;

SELECT * FROM procedure_run_log
WHERE procedure_name = 'deactivate_inactive_customers'
ORDER BY run_date DESC LIMIT 5;

SELECT COUNT(*) AS deactivated_total
FROM customers WHERE is_active = FALSE;
```

Snowflake Scripting version of Practice Problem 2:

sql – Code Example

```
CREATE OR REPLACE PROCEDURE deactivate_inactive_customers_sf()
RETURNS STRING
LANGUAGE SQL
AS
$$
DECLARE
    v_count    INTEGER DEFAULT 0;
    v_cutoff   DATE      DEFAULT DATEADD(DAY, -180, CURRENT_DATE());
    v_msg      STRING;
BEGIN
    -- Insert audit rows
    INSERT INTO customer_deactivation_log(customer_id, deactivated_at,
reason)
    SELECT c.customer_id,
           CURRENT_TIMESTAMP(),
           'No orders since ' || :v_cutoff::STRING
    FROM   customers c
    WHERE  c.is_active = TRUE
           AND NOT EXISTS (
                SELECT 1 FROM orders o
                WHERE  o.customer_id = c.customer_id
                       AND o.order_date ≥ :v_cutoff
            );

    v_count := SQLROWCOUNT;

    -- Deactivate
    UPDATE customers
```

```
SET    is_active = FALSE
WHERE  is_active = TRUE
      AND NOT EXISTS (
          SELECT 1 FROM orders o
          WHERE  o.customer_id = customers.customer_id
              AND o.order_date ≥ :v_cutoff
      );

-- Log
INSERT INTO procedure_run_log VALUES
    ('deactivate_inactive', CURRENT_TIMESTAMP(), 'SUCCESS',
     'Deactivated: ' || v_count);

RETURN 'Deactivated ' || v_count || ' customers';

EXCEPTION
    WHEN OTHER THEN
        LET err_msg := SQLERRM;
        INSERT INTO procedure_run_log VALUES
            ('deactivate_inactive', CURRENT_TIMESTAMP(), 'FAILED',
             :err_msg);
        RAISE;

END;
$$;

CALL deactivate_inactive_customers_sf();
```

SUMMARY CARD

SQL 1.9 Key Takeaways:

- **Function** = returns a value. Used in SELECT/WHERE/expression. No transaction control. No DML side effects.
- **Stored Procedure** = performs actions. Called with CALL. Can COMMIT/ROLLBACK. Can INSERT/UPDATE/DELETE.
- **Scalar function:** one input → one output. Fast for simple calculations. Be aware of per-row overhead on large tables — inline CASE is faster at scale.
- **Table-valued function:** returns a result set. Used in FROM clause. Parameterised view. Cannot be indexed.
- **Stored procedure:** full transaction control. DECLARE variables. IF/ELSIF. Exception handling. RAISE to propagate errors.
- **Snowflake JavaScript:** uses `snowflake` object, `getColumnValue()`. Legacy but widely used.

- **Snowflake Scripting:** SQL-native, DECLARE/BEGIN/END/IF/FOR cursor. Recommended for new procedures.
- **Cursor loops are slow** on large tables — always prefer set-based SQL.
- **Limitations:** hard to version in Git, hard to unit test, tightly coupled to database platform.
- **Use procedures for:** SCD merges, bulk deactivations, audit-logged updates, atomic multi-table workflows.
- **Use dbt for:** all analytical SELECT-based transformations.
- **Always version procedures as .sql files in Git** — never edit directly in the database UI.

🔮 AI DEEP DIVE PROMPT

Copy this prompt into Claude or ChatGPT for deeper practice:

'Act as a senior data engineer. I just learned SQL stored procedures and functions — scalar UDFs, table-valued functions, stored procedures with transaction control, Snowflake Scripting, and when to use procedures vs dbt. Give me 6 progressively harder exercises. Start with a simple scalar function, build to a full Snowflake Scripting procedure with a cursor loop, exception handling, and audit logging. For each exercise: give me the business requirement, ask me to write the function or procedure, then show the correct solution, explain every design decision, point out one common mistake, and give one interview question about that concept. Do not continue until I submit my answer.'

🔗 CHECKPOINT QUESTION

Final review — you are asked to design the stored procedure layer for a Snowflake data warehouse that processes a nightly SCD Type 2 update for the customers dimension. Requirements: detect rows where name, email, or city changed since the last load; expire the old row (set *enddate* = *today*, *iscurrent* = FALSE); insert the new version (with *startdate* = *today*, *enddate* = NULL, *iscurrent* = TRUE); *log a count of changed rows to a dimensionchange_log* table; all operations must be atomic. Write the Snowflake Scripting procedure skeleton — DECLARE block, the MERGE statement logic, the log INSERT, and the EXCEPTION handler. Explain which parts you would NOT put in a procedure and would put in a dbt snapshot instead.

🌟 CHAPTER REVIEW & SUMMARY

What You Learned:

Understanding of the core concepts presented in this chapter including sql 1.9 — stored procedures and functions, their architectural components, practical use cases, and design paradigms.

Quick Revision Notes:

- Key definitions and nomenclature for the chapter topic.
- Hands-on code execution patterns and setup checklists.
- Analogy-driven visualization of data structures or layered setups.

Core Visualized Components:

- **Function vs Stored Procedure — Core Distinction:** A two-column comparison diagram. Left column (blue background, header 'USER-DEFI...